Contextual Chain Protocol Specification

Version 1.0

Song-Ju Kim

April 8, 2026

Document title	Contextual Chain Protocol Specification
Version	1.0
Release date	April 8, 2026
Author	Song-Ju Kim
Document status	Public technical specification and defensive publication
Repository	https://github.com/songju1/Contextual-Chain
Archival DOI	https://doi.org/10.5281/zenodo.19462714
Paper DOI / arXiv	DOI:10.48550/arXiv.260X.XXXX
Document license	Creative Commons Attribution 4.0 International (CC BY 4.0)
Code license	Apache License 2.0
Contact	kim@sobin.org

Abstract

This document specifies *Contextual Chain*, a lightweight ledger protocol for intermittent, noisy, and resource-constrained environments such as mobile and IoT networks. The protocol is organized around two coupled mechanisms: (i) *contextual authentication*, a node-local acceptance and fork-choice rule defined over compact local context rather than full-history validation, and (ii) *adaptive synchronization*, which increases gossip effort only when local inconsistency becomes prevalent.

This specification has two purposes. First, it gives a sufficiently detailed technical description of the base protocol so that an independent implementation can reproduce the operational design used in the accompanying paper. Second, it serves as a public technical disclosure of the protocol family, including extensions that connect synchronization, contextual ambiguity, and memory-burden-based authentication.

A central extension disclosed here is a *memory-burden-oriented proof-of-context* design. Its purpose is not to make the adversary's *computation time* asymptotically dominant, but to make the adversary's required *context-indexed memory burden* grow as multiple plausible contexts must be kept simultaneously alive. This design direction is architecturally motivated by recent Kim results on fixed shared-state semantics and contextual bookkeeping cost [1, 2].

Recommended Citation

If you use or cite this specification, please cite it as:

> Song-Ju Kim. *Contextual Chain Protocol Specification, Version 1.0.* Zenodo, April 8, 2026. DOI: TODO.

Licensing Notice

Public Disclosure Statement

This document is intended to serve as a public technical disclosure of the Contextual Chain protocol family. Its purpose includes establishing a clear public record of the protocol design, its representative embodiments, and its disclosed extension space. In particular, this document discloses:

- the compact-state contextual-authentication base protocol,
- adaptive synchronization under inconsistency,
- checkpoint-based fork choice and quarantine logic,
- contextual-authentication extension families,
- a memory-burden-oriented proof-of-context extension.

Artifact Availability

The following artifacts are intended to accompany this specification:

- reference implementation source code,
- experiment scripts,
- final summary data used in the associated paper,
- figure-generation scripts,
- the accompanying paper and related release notes.

Contents

1 Status, Purpose, and Normative Scope

This document specifies Version 1.0 of the Contextual Chain protocol family.

This specification is intended for two simultaneous purposes:

(1) to define the protocol behavior used in the accompanying paper with enough detail for reproducibility and independent reimplementation;

(2) to provide a public technical disclosure of the protocol design, including its extension space.

The base protocol specified here is **operational** rather than cryptographic. In particular, the term *contextual authentication* in this document means a node-local acceptance rule for choosing a plausible chain head from compact local context. It does **not** mean a complete cryptographic authentication theorem, and it does **not** by itself imply unforgeability, secrecy, or identity binding in the usual cryptographic sense.

The base protocol consists of:

- compact local context maintenance,
- checkpoint-first fork choice,
- inconsistency-driven quarantine,
- adaptive gossip synchronization.

This document also discloses a family of contextual-authentication extensions, including a proof-of-context extension whose core purpose is to impose a growing *memory burden* on an adversary that attempts to remain compatible with multiple concurrent contexts.

2 Source of the Architectural Idea

The design of Contextual Chain is motivated by two different but compatible lines of thought.

First, it is motivated by the systems problem of recovery under intermittent connectivity: in mobile and IoT settings, partitions, delay, and packet loss are not rare anomalies but ordinary operating conditions. In that regime, the central design question is not merely how to validate a full ledger history, but how a resource-constrained node should decide what to accept and how the network should resynchronize after disruption.

Second, the extension space disclosed here is motivated by recent Kim work on fixed shared-state semantics and contextual bookkeeping cost [1, 2]. The central architectural idea is that if a system must preserve context-dependent behavior while reusing a compact shared internal description, then additional *external contextual bookkeeping* may become necessary. In the present protocol family, this viewpoint is operationalized as follows: the base protocol uses compact local context for recovery decisions, while the disclosed proof-of-context extension is designed so that an adversary that wishes to remain compatible with many plausible contexts must bear a growing *memory burden*.

This document therefore treats the Kim results as the source of the architectural perspective and the memory-burden intuition. It does *not* claim that the base protocol itself is a direct theorem instantiation of those papers.

3 Terminology

Node. A participant maintaining a local block graph, a preferred head, and local state used for acceptance and synchronization.

Head. The locally selected preferred tip of a node's current block graph.

Context. The compact local state used by a node to decide which observed chain extension is acceptable. This includes checkpoint metadata, recent proposer history, local inconsistency state, and the current preferred head.

Contextual authentication. The operational acceptance rule by which a node selects a plausible branch relative to its current context.

Checkpoint level. A monotone block metadata field used as the primary fork-choice ordering signal.

Quarantine. A node-local state entered when local inconsistency becomes sufficiently large. In quarantine, head switching becomes more conservative.

Adaptive synchronization. A control mechanism that raises gossip effort when inconsistency becomes sufficiently widespread.

Active context. A context that remains plausible and must still be covered if an entity wishes to appear synchronized with the current network state.

Memory burden. The amount of context-indexed auxiliary memory required to remain compatible with multiple plausible contexts. In this document, the memory burden of the proof-of-context extension is the central resource of interest, not asymptotic CPU time alone.

4 Design Goals

The protocol is designed for environments in which:

(i) nodes are resource-constrained,

(ii) network connectivity is intermittent,

(iii) delay and packet loss are non-negligible,

(iv) partitions and rejoin events are expected,

(v) storing and replaying full ledger history is undesirable.

The Version 1.0 base protocol is intended to achieve the following:

- maintain compact local state,
- recover agreement after partition rejoin,
- reduce oscillatory head switching under local inconsistency,
- provide a synchronization policy that increases effort only when needed,
- expose a structured extension path toward memory-burden-oriented contextual authentication.

5 System Model

5.1 Network Model

The network is modeled as a time-varying communication graph with delay, jitter, packet loss, and optional hard partitions. Messages may be dropped with probability p_{drop} and delivered after random delay sampled from a configured delay model.

The protocol assumes two representative network regimes:

- **clean**: lower delay, zero or negligible loss,
- **noisy**: larger delay and nonzero loss.

5.2 Event-Driven Execution Model

The protocol is specified and evaluated in an **event-driven** simulator. It is **not** a global difference-equation model.

Time advances from one scheduled event to the next. The base event types are:

- `propose`: a node creates a new block,
- `deliver`: a previously transmitted block arrives at a receiver,
- `gossip_tick`: a periodic synchronization step occurs.

Proposal times are sampled stochastically from the target block interval. Delay, jitter, packet loss, and partition windows are external parameters. Node state evolves according to deterministic acceptance and switching rules conditioned on the received events.

5.3 Partition Model

A hard partition divides the nodes into two groups, denoted A and B, over a configured time interval. Messages crossing the partition boundary MUST NOT be delivered during the partition interval. After the interval ends, normal delivery and gossip resume.

Representative split ratios include:

- 50/50,
- 80/20,
- 90/10.

6 Node State

Each node MUST maintain at least the following state:

- a local block graph, including parent links and known children;
- a current preferred head h;
- per-block checkpoint metadata:
  - checkpoint level cp_level,
  - checkpoint hash cp_hash;
- a proposer reputation table $R[\cdot]$;

- equivocation records keyed by proposer and height;
- recent-window statistics for:
  - fork pressure,
  - reorganization magnitude,
  - equivocation count;
- a smoothed inconsistency estimate $\overline{\mathcal{I}}_t$;
- a binary quarantine flag $Q \in \{0, 1\}$.

The protocol is explicitly compact-state. Nodes are not required to replay or store the full global history before making local head-selection decisions.

7 Block Metadata and Basic Objects

Each block MUST contain at least:

- a unique block identifier,
- a parent identifier,
- a proposer identifier,
- a height,
- a checkpoint level,
- a checkpoint hash.

The checkpoint hash is retained as metadata for compatibility with stronger checkpointing and authentication extensions. In Version 1.0 of the base protocol, the primary checkpoint signal used by fork choice is the checkpoint *level*.

8 Checkpointing

8.1 Height-Based Checkpointing

Let E denote the checkpoint epoch length in blocks. When a proposer extends its current head to height t, the checkpoint level is updated by

$$\text{cp\_level}(t) = \begin{cases} \text{cp\_level}(t-1) + 1 & \text{if } t \bmod E = 0, \\ \text{cp\_level}(t-1) & \text{otherwise.} \end{cases}$$

8.2 Time-Based Checkpointing

An implementation MAY alternatively define checkpoint epochs by elapsed time rather than by height. In that case, checkpoint level increments at configured wall-clock epoch boundaries.

8.3 Sticky Checkpoint Tie-Breaking

A checkpoint-hash-based sticky tie-break was considered during development. It is explicitly **not part of the normative Version 1.0 base configuration**.

9 Fork Choice and Contextual Authentication

9.1 Primary Fork-Choice Rule

Given the visible tip set $\mathcal{T}$, a node MUST select a candidate head $h^\star$ by lexicographic priority:

$$h^\star = \arg\max_{t\in\mathcal{T}} \Big(\text{cp\_level}(t),\ \text{height}(t),\ \text{branch\_score}(t)\Big),$$

with deterministic tie-breaking.

The ordering priority is therefore:

$$\text{higher checkpoint level} \ \rightarrow\ \text{higher height} \ \rightarrow\ \text{higher branch score}.$$

This rule is the core of contextual authentication in the base protocol: the node decides which chain extension is locally acceptable using compact contextual state rather than full-history validation.

9.2 Branch Score

For a tip t, let $\text{tail}_L(t)$ denote the last L blocks on the chain to t. The node computes

$$\text{branch\_score}(t) = \sum_{b\in\text{tail}_L(t)} \log\big(1+\max\{0, R[p(b)]\}\big),$$

where $p(b)$ is the proposer of block b.

This score:

- favors branches recently extended by better-behaving proposers,
- remains local and lightweight,
- damps individual contributions through the logarithm.

10 Equivocation Handling

If a node observes two different block identifiers from the same proposer at the same height, it MUST record an equivocation event.

In the Version 1.0 base protocol, the node applies the local penalty

$$R[p] \leftarrow \max\{0,\ R[p]-1.0\}.$$

Equivocation events MUST also be inserted into the recent inconsistency window.

11 Inconsistency Estimation

11.1 Snapshot Inconsistency Score

At each block acceptance, the node computes a snapshot inconsistency score from recent local statistics.

Let:

- forktop be the number of tips at the current maximum height,
- reorg_recent be the maximum reorganization magnitude in the recent window,
- equiv_recent be the number of recent equivocation events.

The snapshot score is

$$\mathcal{I}_t = 1.2\log(1+\text{forktop}) + 0.7\sqrt{\text{reorg\_recent}} + 0.9\log(1+\text{equiv\_recent}).$$

11.2 EMA Update

The node updates the inconsistency EMA by

$$\overline{\mathcal{I}}_t = \alpha \overline{\mathcal{I}}_{t-1} + (1-\alpha)\mathcal{I}_t.$$

The default parameter is

$$\alpha = 0.85.$$

12 Quarantine

12.1 Hysteresis Rule

Let T_{on}, T_{off}, and S denote:

- the quarantine entry threshold,
- the quarantine exit threshold,
- the number of consecutive below-threshold updates required for exit.

The node enters quarantine if

$$\overline{\mathcal{I}}_t > T_{\text{on}}.$$

The node exits quarantine only after

$$\overline{\mathcal{I}}_t < T_{\text{off}}$$

for S consecutive updates.

The default values are

$$T_{\text{on}} = 1.05, \qquad T_{\text{off}} = 0.75, \qquad S = 25.$$

12.2 Meaning of Quarantine

Quarantine is not a complete repair mechanism. Its role is to reduce oscillatory head switching under local inconsistency. Because quarantine alone does not restore missing information, it is coupled with adaptive synchronization.

13 Head Switching Policy

13.1 Outside Quarantine

If $Q = 0$, the node SHOULD switch immediately to the current fork-choice winner $h^\star$.

13.2 Inside Quarantine

If $Q = 1$, switching MUST be more conservative:

- switch immediately if the candidate has higher checkpoint level;
- if checkpoint levels match, switch if the candidate improves height by at least 1;
- if heights also match, switch only if the candidate branch score exceeds the current head by at least 0.15.

This rule is intended to reduce oscillation without freezing progress.

14 Reputation Reward

When a block is accepted and no equivocation penalty applies, the proposer receives a small positive update:

$$R[p] \leftarrow R[p] + 0.05.$$

15 Adaptive Synchronization

15.1 Gossip Mechanism

At every gossip tick, provided the network is not currently partitioned, the protocol:

1. samples sender/receiver pairs,
2. identifies the highest common ancestor between the receiver and the sender's head chain,
3. transfers the missing suffix from sender to receiver in order.

This gossip step is distinct from ordinary per-block broadcast. Its purpose is to repair missing information after divergence.

15.2 Adaptive Gossip Budget

Let q be the fraction of nodes currently in quarantine. If

$$q \geq 0.25,$$

the protocol uses an increased gossip budget. Otherwise it uses the normal budget.

In the base evaluation profile,

$$\text{normal gossip pairs} = 1, \qquad \text{quarantine gossip pairs} = 4.$$

16 Normative Evaluation Variants

The following protocol variants are defined for evaluation and comparative disclosure:

Variant	Definition
`NoQ`	Quarantine disabled; gossip fixed at the normal budget.
`Q_only`	Quarantine enabled; gossip fixed at the normal budget.
`Gossip_only`	Quarantine disabled; gossip fixed at an aggressive budget.
`Both`	Quarantine enabled; gossip uses the normal budget by default and increases when quarantine prevalence exceeds threshold.

These variants are part of the public disclosure because they define the operational comparison space of the protocol family.

17 Event Semantics

17.1 Propose Event

On a `propose` event, a node:

1. extends its current head,

2. creates a new block,
3. assigns checkpoint metadata,
4. schedules delivery attempts to reachable peers subject to delay and drop.

17.2 Deliver Event

On a `deliver` event, the receiver node:

1. ignores the block if already known,
2. stores the block as an orphan if the parent is unknown,
3. otherwise executes the acceptance procedure of Section 18.

17.3 Gossip Tick Event

On a `gossip_tick` event, sender/receiver pairs are sampled and missing suffixes are transferred according to the currently active gossip budget.

18 Acceptance Procedure

Upon receipt of a block b with parent pointer b.prev, a node MUST process the block as follows:

1. If b is already known, return.
2. If b.prev is unknown, store b as an orphan and return.
3. Detect equivocation for $(b.\text{proposer}, b.\text{height})$.
4. Update local reputation and recent equivocation statistics.
5. Insert b into the local block graph.
6. Compute the inconsistency snapshot $\mathcal{I}_t$.
7. Update the EMA $\overline{\mathcal{I}}_t$.
8. Update the quarantine flag by hysteresis.
9. Compute the candidate head $h^\star$ via checkpoint-first fork choice.
10. If quarantine is active, apply the conservative switching rule; otherwise set $h \leftarrow h^\star$.
11. If the head changed, record reorganization magnitude in the recent window.
12. Apply the proposer reward if no equivocation penalty was triggered.

19 Evaluation Metrics

The primary metrics are:

Success rate. The fraction of runs in which all nodes end with the same head.

Recovery time. The time from partition end (rejoin) to the first detected convergence event.

Recovery tail. Typically the recovery $p95$ in addition to mean recovery.

Fork and reorganization summaries. Auxiliary statistics on divergence and switching.

Synchronization effort. Measured by mean gossip-pair usage and related effort proxies.

Estimated communication volume. A block-transfer-based byte estimate. This is a proxy rather than a packet-level trace.

20 Reference Experimental Profile

A representative evaluation profile for the Version 1.0 base protocol is:

- $N = 20$ nodes for the main suite,
- simulation horizon = 3600 s,
- target global block interval = 30 s,
- partition interval $[1200, 2400]$ s,
- checkpoint epoch length = 30 blocks,
- convergence window $K_{\text{CONVERGE}} = 30$,
- clean and noisy network regimes,
- 50/50 and 80/20 primary partition scenarios,
- 90/10 robustness scenario.

This reference profile is *representative*, not exclusive. Alternative parameterizations are also part of the disclosed design space.

21 Contextual-Authentication Extension Family

The Contextual Chain family includes several contextual-authentication extensions. These are disclosed here in order to broaden the public technical record and to make explicit that the protocol family is not limited to the base operational recovery rule.

21.1 Extension A: Context-Suffix Challenge

A verifier samples heights or block positions in the suffix after the latest checkpoint. A peer is challenged to return the corresponding block headers or parent-hash evidence. Acceptance depends on sufficient consistency with the verifier's current suffix.

This extension is lightweight and directly connected to synchronization status after rejoin.

21.2 Extension B: Memory-Burden-Oriented Proof of Context

This extension is the primary extension of interest in Version 1.0. Its core purpose is to make contextual compatibility expensive in *memory burden*, not merely in CPU cycles.

21.2.1 Architectural Goal

The goal is to construct a contextual-authentication layer in which:

- an honest synchronized participant needs to maintain only the currently relevant context,
- an adversary wishing to remain compatible with many plausible contexts must maintain multiple context-indexed auxiliary objects,
- the dominant adversarial burden is the memory needed to keep these contextual objects available.

This extension is the clearest operational analogue of the Kim fixed-shared-state bookkeeping viewpoint [1, 2].

21.2.2 Abstract Interface

Let:

- S_t be the compact shared state at time t,
- C_t be the current interaction context,
- O_{t-1} be the previous accepted outcome or checkpoint summary.

Define the context-dependent seed

$$\mathsf{seed}_{t,c} = H(S_t \,\|\, O_{t-1} \,\|\, c), \qquad c \in \mathcal{X}_t,$$

where $\mathcal{X}_t$ is the set of still-plausible contexts at time t.

For each candidate context c, the prover derives a context-indexed memory object

$$A_{t,c} = \mathsf{Build}(\mathsf{seed}_{t,c}),$$

and a short commitment

$$R_{t,c} = \mathsf{Commit}(A_{t,c}).$$

Given a challenge $\mathsf{chal}_{t,c}$ for context c, the prover returns

$$\pi_{t,c} = \mathsf{Open}(A_{t,c}, \mathsf{chal}_{t,c}),$$

and the verifier accepts if

$$\mathsf{Verify}(R_{t,c}, \mathsf{chal}_{t,c}, \pi_{t,c}) = 1.$$

The functions above are intentionally abstract interfaces:

- Build constructs a context-indexed memory object,
- Commit maps that object to a short commitment,
- Open returns a proof for a specific challenge,
- Verify checks consistency of proof, commitment, and challenge.

21.2.3 Memory Burden Rather Than Computation Burden

The key design intent is not merely to increase computation time. Instead, the main burden is the need to *store* enough context-indexed information to answer correctly across many plausible contexts.

Let $c_t^\star$ denote the honest current context. Then the honest participant needs only

$$\mathrm{Mem}_{\mathrm{hon}}(t) \approx |A_{t,c_t^\star}|.$$

By contrast, if an adversary wishes to remain compatible with a covered subset $\mathcal{Y}_t \subseteq \mathcal{X}_t$, then the adversary's stored burden is at least

$$\mathrm{Mem}_{\mathrm{adv}}(t) \gtrsim \sum_{c \in \mathcal{Y}_t} |A_{t,c}| + H(M_t),$$

where M_t is any additional contextual index required to disambiguate among the covered contexts.

The term $H(M_t)$ is included to make explicit the Kim-style contextual bookkeeping interpretation: if a single compact shared state must support multiple context-dependent responses, some additional contextual indexing burden is generally required.

A minimal bookkeeping inequality is therefore

$$H(M_t) \gtrsim \log |\mathcal{X}_t|$$

whenever the adversary must keep responses separable across the active context set $\mathcal{X}_t$.

21.2.4 Challenge Coverage Rule

Suppose the verifier samples k contexts uniformly without replacement from $\mathcal{X}_t$ and requires the prover to answer for all of them. If the adversary stores objects for only B_t covered contexts, where $B_t < |\mathcal{X}_t|$, then under uniform sampling the adversary's acceptance probability is upper bounded by

$$\Pr[\mathrm{accept}_t] \le \frac{\binom{B_t}{k}}{\binom{|\mathcal{X}_t|}{k}} \qquad \text{for } B_t \ge k,$$

and

$$\Pr[\mathrm{accept}_t] = 0 \qquad \text{for } B_t < k.$$

This bound makes the memory-burden story operational. The adversary does not fail because it cannot do enough arithmetic quickly enough; it fails because it has not kept enough contexts alive in memory.

21.2.5 Why the Burden Grows Over Time

As partition, delay, and equivocation accumulate, the set of plausible contexts need not remain small. If multiple context trajectories remain live over several rounds, then:

- the active context set $|\mathcal{X}_t|$ can grow,
- the context-indexed memory family $\{A_{t,c}\}_{c \in \mathcal{X}_t}$ becomes larger,
- the auxiliary indexing burden M_t also grows.

Thus the extension is explicitly designed so that adversarial compatibility with many concurrent contexts becomes increasingly expensive in memory.

21.2.6 Representative Simplified Experimental Form

The current paper evaluates a simplified operational form of Extension B rather than a fully cryptographic instantiation. In that simplified form:

- the attacker is assigned a bounded context budget,
- the challenge samples multiple active contexts,
- success depends on whether those contexts are covered.

The key recorded quantities include:

- number of required contexts at peak,
- number of required contexts at rejoin,
- honest-node peak memory,
- adversarial stored memory,
- required peak memory for full active-context coverage.

This simplified extension is not a theorem-level cryptographic protocol. It is a concrete operational disclosure of a memory-burden-oriented contextual-authentication design.

21.3 Extension C: Checkpoint Attestation

A checkpoint may be accompanied by an explicit signed attestation from a proposer or authorized participant. This moves the protocol closer to a signature-based design and is therefore outside the narrow operational base scope of Version 1.0. It is nevertheless disclosed here as part of the extension family.

21.4 Extension D: Quarantine Handshake

When a peer is suspected or when quarantine is active, the protocol may require the peer to provide additional synchronization evidence, such as:

- missing suffix fragments,
- competing-tip lists,
- recent equivocation evidence,
- checkpoint-consistency evidence.

This extension is operational rather than cryptographic, but it is part of the disclosed family.

22 Interpretation of the Base Protocol and the Extensions

The base protocol of Version 1.0 should be interpreted as:

- a lightweight recovery protocol,
- a compact-state acceptance mechanism,
- a synchronization-control policy for partition/rejoin environments.

The extension family should be interpreted as:

- a set of disclosed contextual-authentication directions,
- including a memory-burden-oriented proof-of-context extension,
- without claiming that Version 1.0 already provides a complete cryptographic authentication theorem.

23 Implementation and Measurement Checklist

A minimally conforming implementation of the Version 1.0 base protocol SHOULD provide:

1. an event-driven execution loop,
2. stochastic proposal timing,
3. delayed and lossy delivery,
4. partition/rejoin support,
5. local block graph maintenance,
6. checkpoint metadata updates,
7. branch score computation,
8. equivocation detection and local penalty,
9. inconsistency EMA and hysteresis,
10. quarantine-aware head switching,
11. periodic gossip and adaptive gossip budget control,
12. measurement of success, recovery, reorganization, and synchronization effort.

A conforming experimental implementation of the simplified proof-of-context extension SHOULD additionally provide:

1. a finite attacker context budget,
2. a configurable active-context challenge rule,
3. tracking of required active contexts,
4. tracking of honest and adversarial memory footprints,
5. reporting of challenge success, rejoin success, and end success.

24 Version 1.0 Limitations

The Version 1.0 protocol family has the following known limitations:

1. synchronization-cost quantities are still simulator-side proxies rather than implementation-level energy or packet-trace measurements;
2. convergence is operational rather than theorem-level finality;
3. the base protocol is not yet a completed large-scale solution at arbitrary network size;
4. the proof-of-context extension is disclosed in both abstract and simplified forms, but not yet as a full deployed cryptographic construction.

25 Defensive-Publication Intent

This document is intended to serve as a public technical disclosure of the Contextual Chain protocol family, including:

- the compact-state contextual-authentication base protocol,
- the adaptive synchronization mechanism,
- the checkpointing and quarantine rules,
- the family of contextual-authentication extensions,
- in particular the memory-burden-oriented proof-of-context extension.

The disclosed subject matter includes both the specific representative forms described in the accompanying implementation and broader equivalent embodiments consistent with the same architectural principles: compact local acceptance, context-indexed ambiguity management, adaptive synchronization under inconsistency, and memory-burden-based contextual coverage.

Scope Note

This document is a technical protocol specification and public disclosure. It is not itself a standards-track interoperability RFC, and it does not claim a complete cryptographic security model. Its purpose is to specify the Version 1.0 design and to place the core protocol ideas and extension space into the public technical record.

References

[1] Song-Ju Kim. Contextuality as an external bookkeeping cost under fixed shared-state semantics. *arXiv preprint arXiv:2601.20167*, 2026.

[2] Song-Ju Kim. Contextuality from single-state ontological models: An information-theoretic no-go theorem. *arXiv preprint arXiv:2602.16716*, 2026.